

Machine Learning 2020

Practice Exam A

WITH ANSWERS

March 21, 2023

This document provides a detailed explanation of the exam structure with some example questions for each category. It is *not* an example of what the actual exam will look like. For that, see Practice Exam B.

Some tips:

- Pay particular attention to **the application questions**. These are the most difficult, but they are also the most *predictable*. If you have a look at them early on, you should know what to pay attention to as the course progresses.
- The exam contains easy questions as well as tricky ones. Don't make the mistake of suspecting trick questions when something seems too good to be true. Some questions are just easy.
- Focus on revising for the **recall questions** and the **application questions**. If you practice these well, the exam should be easy to pass.
- Note that application questions follow a fixed pattern. If you run out of practice questions, you can easily create your own examples.
- The exam is **only two hours**. To get a good grade you should practice enough to answer questions quickly as well as accurately.

1 Recall questions

Approximately one third of the exam will be *recall questions*. These are questions that ask for a simple detail from a single slide without too much depth. If you have seen and understood every slide of every lecture once, you should be able to answer the majority of recall questions correctly. These questions are never phrased to trick you or to catch you out.

Examples of recall questions

1. What separates **offline learning** from **reinforcement learning**?
 - A In reinforcement learning the training labels are reinforced through boosting.
 - B Offline learning can be done without connection to the internet. Reinforcement learning requires reinforcement from a separate server.
 - C In reinforcement learning, the learner takes actions and receives feedback from the environment. In offline learning we learn from a fixed dataset. ✓
 - D Reinforcement learning uses backpropagation to approximate the gradient, whereas offline learning uses symbolic computation.

See lecture 1. Note that the incorrect questions are phrased to sound reasonable if you don't know the specifics, but are mostly nonsense, if you know what these concepts mean.
2. The most important rule in machine learning is “never judge your performance on the training data.” If we break this rule, what can happen as a consequence?
 - A The loss surface no longer provides an informative gradient.
 - B We get cost imbalance.
 - C We end up choosing a model that overfits the training data. ✓
 - D We commit multiple testing.

Introduced in lecture 1, also discussed in lecture 3.
3. We have a classifier **c** and a test set. Which is **true**?
 - A To compute the *precision* for **c** on the test set, we must define how to turn it into a ranking classifier.
 - B To compute the *false positive rate* for **c** on the test set, we must define how to turn it into a ranking classifier.
 - C To compute the *confusion matrix* for **c** on the test set, we must define how to turn it into a ranking classifier.
 - D To compute the ROC *area under the curve* for **c** on the test set, we must define how to turn it into a ranking classifier. ✓

The first three can be computed for any binary classifier. For the ROC AUC, however, we need a ranking of the instances in the test set from

most **negative** to most **positive**. See lecture 3 and homework 3.

4. Testing too many times on the test set increases the chance of random effects influencing your choice of model. Nevertheless, we may need to test many different models and many different hyperparameters. What is the solution suggested in the lectures?
- A To withhold the test set, and use a train/validation split on the remainder to evaluate model choices and hyperparameters. ✓
- B To normalize the the data so that they appear normally distributed. **Normalization will not help with this problem.**
- C To use bootstrap sampling to gauge the variance of the model. **Bootstrap sampling (lecture 3 and lecture 10) will help you gauge the variance. But that will not solve this problem.**
- D To use a boosted ensemble, to reduce the variance of the model, and with it, the probability of random effects. **An ensemble (lecture 10) can increase performance by reducing bias and variance, but this does not help with the problem of test-set reuse (lecture 3). It's just another way of building a model.**
- This is discussed in lecture 3. It is a crucial part of any machine learning project.**

2 Combination questions

Approximately one third of the exam will be *combination questions*. These are slightly deeper than simple recall questions. They may, for instance, require you to

- combine pieces of information from different parts of the lecture,
- Answer a question posed in the negative, i.e. “Which is *not* one of the reasons that . . . ?”,
- recognise that an answer that seems correct is actually not true. We won't write trick question for the sake of tricking you, but sometimes it's important to include common misconceptions.

The difference between recall and combination questions is a little fuzzy, but hopefully, you can infer the general idea from the examples below.

Examples of combination questions

5. Different features in our data may have wildly different scales: a person's age may fall in the range from 0 to 100, while their savings can

fall in the range from 0 to 100 000. For many machine learning algorithms, we need to modify the data so that all features have roughly the same scale. Which is **not** a method to achieve this?

- A Imputation ✓
- B Standardization
- C Normalization
- D Principal Component Analysis

This question is phrased negatively. This usually makes it more difficult to guess. If you know that imputation is a way to deal with missing data (not normalization), you can get the answer that way. To get the answer by elimination, you'll have to know that B, C and D can all be used for normalization of the features. D is a tricky one, since PCA is more commonly used for dimensionality reduction, but it can also be used for normalization.

6. The *variational* autoencoder adapts the regular autoencoder in a number of ways. Which is **not** one of them?
- A It adds a sampling step in the middle.
 - B It makes the outputs of the encoder and decoder parameters of probability distributions.
 - C It adds a loss term to ensure that the latent space is laid out like a standard normal distribution.
 - D It adds a discriminator that learns to separate generated examples from those in the dataset. ✓

Again, a negative question. You'll need to know all the differences between an autoencoder and a VAE in order to get the answer, or you need to recognize that a discriminator is component of a GAN, not a VAE.

7. We have two discrete random variables: A with outcomes 1, 2, 3 and B with outcomes a, b, c . We are given the joint probability $p(A, B)$ in a table, with the outcomes of A enumerated along the rows (vertically), and the outcomes of B enumerated along the columns (horizontally). How do we compute the probability $p(A = 1 \mid B = a)$?
- A We find the probability in the first column and the first row.
 - B We find the probability in the first column and the first row, and divide it by the sum over the first column. ✓
 - C We find the probability in the first column and the first row, and divide it by the sum over the first row.
 - D We sum the probabilities over the first column and the first row.

The answer to this question resides in a single slide (in lecture 5), but you'll have to have remembered the concept of conditional probability well enough to transform the information here to the correct picture.

8. Why is gradient descent difficult to apply in a reinforcement learning setting?
- A The loss surface is flat in most places, so the gradient is zero almost everywhere.
 - B The backpropagation algorithm doesn't apply if the output of a model is a probability distribution.
 - C There is a non-differentiable step between the model input and the model output.
 - D There is a non-differentiable step between the model output and the reward. ✓

This information is again contained in a single point in the lecture series, but it's at the end (in the 13th lecture) and not extensively discussed.

9. The squared error loss function looks like this: $\sum_i (y_i - t_i)^2$, where the sum is over all instances, y_i is the model output for instance i and t_i is the training label. Which is **not** a reason for squaring the difference between the two?
- A It ensures that negative and positive differences don't cancel out in the sum.
 - B It ensures that large errors count very heavily towards the total loss.
 - C When used in classification, it ensures that points near the decision boundary weigh most heavily. ✓
 - D It is a consequence of assuming normally distributed errors, and deriving the maximum likelihood solution.

Again, a negative question. Answers A and B were discussed in lecture 1 and repeated in lecture 2. Answer D was discussed in the second probability lecture. Answer C, however, is the opposite of what the least squares loss does. It is true for the log loss.

3 Application questions

The final third of the exam will be *application questions*. These are questions that ask you to apply an algorithm, perform some computation or follow some derivation. These are the questions which you'll need to actively practice for.

All application questions follow a predetermined pattern and are practiced in the homework exercises.

There are 10 types, with a sequence of about three questions for each type. For each exam we will select some types, and adapt the specifics of the question but not the structure. For instance, we may change the dataset or change which parameter we take a derivative for.

The following is a **complete list** of all types. If you master all 10 types given below, there will be no surprises on the exam.

1. **Find the gradient** For a simple (usually polynomial) model, work out the derivative with respect to one of the parameters.
2. **Find a ranking** Given a simple dataset, and a linear classifier work out a ranking of the instances, and identify the number of ranking errors and the coverage.
3. **Entropy** For a given set of probability distributions, compute the entropy and the cross-entropy.
4. **Scalar backpropagation** Apply the backpropagation algorithm to a complicated scalar function. Break the function up into modules and use the local derivatives to compute the derivative for a particular input.
5. **Decision trees** Given a dataset, work out which feature makes for the best split.
6. **Evidence lower bound** Work through the derivation of the *evidence lower bound* (as used in the EM and VAE algorithms) and identify the missing steps.
7. **Naive Bayes** Given a dataset with categorical variables, compute the probabilities that a naive Bayes classifier assigns to each class, with and without smoothing.
8. **Support vector machines** Given a linear classifier trained as an SVM, find the support vectors, and how points are classified.
9. **Markov models** Similar to the *Naive Bayes* category, but with a first-order Markov model.
10. **Matrix backpropagation** For a given module in a matrix-based automatic differentiation system, work out the Jacobian/vector products required to implement the backward pass.

In some cases there are questions included to check that you understand *what* you're doing and what it means, in addition to knowing *how* to do it. These will be entirely different in the exam.

Examples of application questions

type: find the gradient

We want to train the following model:

$$y_i = -vx_i^2 + wx_i + b$$

with parameters v , w and b , where x_i and y_i are scalars. the dataset provides target values t_i . We derive the gradient of the loss with respect to b as follows:

$$\frac{\partial \frac{1}{2} \sum_i (y_i - t_i)^2}{\partial b} = \frac{1}{2} \frac{\partial \sum_i (y_i - t_i)^2}{\partial b} \quad (1)$$

$$= \frac{1}{2} \sum_i \frac{\partial (y_i - t_i)^2}{\partial b} \quad (2)$$

$$= \frac{1}{2} \sum_i \frac{\partial (y_i - t_i)^2}{\partial (y_i - t_i)} \frac{\partial (y_i - t_i)}{\partial b} \quad (3)$$

$$= \frac{1}{2} \sum_i 2 (y_i - t_i) \frac{\partial (y_i - t_i)}{\partial b} \quad (4)$$

10. To get from line (2) to line (3), we use the

- A Chain rule ✓
- B Product rule
- C Constant factor rule
- D Sum rule

11. To get from line (1) to line (2), we use the

- A Chain rule
- B Product rule
- C Constant factor rule
- D Sum rule ✓

The names of the rules are given on your cheat sheet so you don't have to memorize them. Just make sure that you understand what is happening in steps like these, and you can look up the answer.

12. Fill in the definition of y_i and work out the derivative with respect to

b . Which is the correct result?

- A $\frac{1}{2} \sum_i (-vx_i^2 + wx_i + b - t_i)$
- B $\sum_i (-vx_i^2 + wx_i + b - t_i)$ ✓
- C $\frac{1}{2} \sum_i x_i (-vx_i^2 + wx_i + b - t_i)$
- D $\sum_i x_i (-vx_i^2 + wx_i + b - t_i)$

Note that the $\frac{1}{2}$ in front of the loss is canceled out by the 2 factor we received in step 4. For more practice try taking the derivative wrt to w . You should get answer D.

type: find a ranking

We have the following training set:

	x_1	x_2	label
a	0	1	Neg
b	2	2	Neg
c	1	4	Neg
d	2	5	Neg
e	3	6	Pos
f	6	8	Pos
g	5	3	Pos
h	8	7	Pos

For the following questions, it helps to draw the data and the classification boundary in feature space.

We use a linear classifier defined by

$$c(x_1, x_2) = \begin{cases} \text{Pos} & \text{if } 0 \cdot x_1 + x_2 - 2 > 0 \\ \text{Neg} & \text{otherwise.} \end{cases}$$

Note that this classifier ignores x_1 , so we can make things easy for ourselves by just looking at x_2 . The bigger x_2 , the more positive the class (careful, this may be the other way around in other questions), so we can just sort the instances by x_2 .

It will never be more difficult than this on the exam, but make sure you know how you'd do this with multiple features, or with a decision tree classifier.

13. If we turn c into a *ranking* classifier, how does it rank the points, from most **Negative** to most **Positive**?

A a b c d e f g h

B a b g c d e h f ✓

C a b g c d h e f

D a c b d e g f h

See lecture 3 and homework 3. For the linear classifier, we use the distance to the decision boundary to rank the points.

14. How many ranking errors does the classifier make?

A None

B 1

C 2 ✓

D 3

This question is a *very* common pitfall for students. Make sure you understand what a *ranking error* is. Here's a hint: on a dataset of 5 instances, a classifier can make as many as 10 ranking errors.

15. If we draw a coverage matrix (as done in the slides), what proportion of the cells will be red?

- A $\frac{3}{15}$ B $\frac{6}{15}$ C $\frac{1}{8}$ ✓ D $\frac{6}{16}$

In the coverage matrix, each cell is a pair of instances, consisting of a positive and a negative pair (all pairs that might cause a ranking error). This question asks for the proportion of ranking errors made (see the previous question) to the total proportion of *possible* ranking errors. Note that in this case we do simplify the fraction from $2/16$ to $1/8$.

type: Entropy

Here are two distributions, p and q , on the members of a set $X = \{a, b, c, d\}$. We will use binary entropy (i.e. computed with base-2 logarithms). Note that in the computation of the entropy $0 \cdot \log_2(0) = 0$, but otherwise, $\log_2 0$ is undefined as usual.

	p	q
a	$1/4$	0
b	$1/4$	$1/4$
c	$1/4$	$1/4$
d	$1/4$	$1/2$

16. What are their entropies?
- A $H(p) = 2, H(q) = 1$
- B $H(p) = 2, H(q) = 1.5$ ✓
- C $H(p) = 1, H(q) = 1$
- D $H(p) = 1, H(q) = 1.5$
17. What is the cross-entropy $H(q, p)$?
- A $H(q, p) = 1$
- B $H(q, p) = 1.5$
- C $H(q, p) = 2$ ✓
- D $H(q, p) = 2.5$

These two questions are basic application of the entropy formulas on your cheat sheet. However, to calculate this quickly, it can help to write out the sum as logarithms, and apply what you know about logarithms to simplify the sum, before you grab the calculator. For

instance:

$$\begin{aligned} -H(\mathbf{q}) &= 0 \log 0 + \frac{1}{4} \log \frac{1}{4} + \frac{1}{4} \log \frac{1}{4} + \frac{1}{2} \log \frac{1}{2} \\ &= \frac{2}{4} \log \frac{1}{4} + \frac{2}{4} \log \frac{2}{4} \\ &= \frac{2}{4} (\log \frac{1}{4} + \log \frac{2}{4}) \\ &= \frac{2}{4} (-\log 4 + \log 2 - \log 4) \\ &= \frac{2}{4} (-2 + 1 - 2) = -1.5 \end{aligned}$$

It's extremely easy to make sign errors when dealing with entropy. Computing $-H$ makes the formula a positive sum, which makes things a little simpler.

With a bit of practice, this sort of thing is much easier (and more accurate) than punching the formula into the calculator explicitly.

18. If you try to compute $H(\mathbf{p}, \mathbf{q})$, you'll notice that something goes wrong. What does this mean?
- A As $q(a)$ goes to zero, a 's codelength with code $\mathbf{q}$ goes to infinity. This makes the expected codelength under the uniform distribution $\mathbf{p}$ infinite as well. ✓
 - B Because $\mathbf{p}$ is uniform, its expected codelength is always optimal under any distribution.
 - C A standard (graphical) calculator does not have the precision to compute the answer. In a python notebook, we would not have a problem.
 - D Because $q(a) = 0$ the expected codelength with code $\mathbf{q}$, under distribution $\mathbf{q}$ is not defined. This means that the resulting cross-entropy also becomes undefined.

We will occasionally throw in questions like this to test your understanding of a formula. This is a particularly tough one: it requires you to understand the intuitive explanation of entropy in lecture 5, and how it leads to the formula for entropy.

Specifically, the correspondence between codelength and probability: the lower the probability, the higher the codelength. If the probability is 0, the codelength becomes infinite. This wasn't problem for $H(\mathbf{q})$, because $0 \log 0$ is defined to be 0 in the context of entropy, but for the cross-entropy, we end up with an infinity.

type: Backpropagation

NB: This question type is not just asking for a derivative (although the function may be simple enough for that). What we want is for you to apply the

backpropagation algorithm: that is, to work out the local derivatives symbolically, and then do the rest numerically.

We will use the backpropagation algorithm to find the derivative of the function

$$f(x) = \sin(\sin(x) \cos(x))$$

with respect to x .

First, we break the function up into modules:

$$f = \sin(c)$$

$$c = ab$$

$$b = \cos(x)$$

$$a = \sin(x)$$

19. What should be in the place of the dots?

A $b = \cos(x)$, $c = ab$ ✓

B $b = \cos(x)$, $c = \sin(x) \cos(x)$

C $b = \cos(c)$, $c = ab$

D $b = \cos(c)$, $c = \sin(x) \cos(x)$

20. For the backpropagation algorithm, we need to work out the local derivatives symbolically. Which are the required local derivatives?

A $\partial f / \partial c$, $\partial c / \partial a$, $\partial c / \partial b$, $\partial a / \partial x$ and $\partial b / \partial x$ ✓

B $\partial c / \partial f$, $\partial a / \partial c$, $\partial b / \partial c$, $\partial x / \partial a$ and $\partial x / \partial b$

C $\partial f / \partial c$, $\partial f / \partial a$, $\partial f / \partial b$ and $\partial f / \partial x$

D $\partial c / \partial f$, $\partial a / \partial f$, $\partial b / \partial f$ and $\partial x / \partial f$

21. In terms of the local derivatives. Which is the correct expression for the gradient $\partial f / \partial x$?

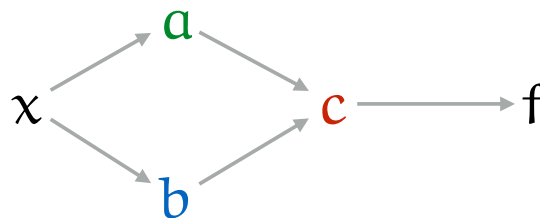
A $\sin(c) [b \cos(x) + a \sin(x)]$

B $\sin(c) [b \cos(x) - a \sin(x)]$

C $\cos(c) [b \cos(x) + a \sin(x)]$

D $\cos(c) [b \cos(x) - a \sin(x)]$ ✓

The most difficult part of this question type is (usually) the application of the *multivariate chain rule* (see the Deep Learning 1 lecture): this rule tells us how the chain rule works if the variable for which we are taking the derivative (in this case x) affects the output (f) along multiple paths. It can help to draw a diagram of the computation graph over the modules:



The multivariate chain rule says that because x influences f along two paths along the computation graph, we can take the derivatives along both paths and sum them. The variables that on the other paths are taken as constants. In this cases:

$$\begin{aligned}
 \frac{\partial f}{\partial x} &= \frac{\partial f}{\partial c} \frac{\partial c}{\partial x} && \text{application of the basic chain rule} \\
 &= \frac{\partial f}{\partial c} \left(\frac{\partial c}{\partial a} \frac{\partial a}{\partial x} + \frac{\partial c}{\partial b} \frac{\partial b}{\partial x} \right) && \text{mv chain rule to simplify } \frac{\partial c}{\partial x} \\
 &= \frac{\partial f}{\partial c} \frac{\partial c}{\partial a} \frac{\partial a}{\partial x} + \frac{\partial f}{\partial c} \frac{\partial c}{\partial b} \frac{\partial b}{\partial x} && \text{simplify}
 \end{aligned}$$

We can now work out the local derivatives for each module, fill them in, and simplify. See homework exercise 5 for the rest of the process.

type: Decision trees

Consider the following task. The aim is to predict the class y from the binary features x_1 , x_2 , x_3 and x_4 .

x_1	x_2	x_3	x_4	y
B	A	A	A	Yes
A	A	B	A	No
B	A	A	A	Yes
A	A	B	A	No
B	B	A	A	Yes
B	B	A	A	Yes
B	A	A	B	Yes
A	B	B	B	No
A	A	B	B	No
A	B	A	B	Yes
B	B	B	B	No
A	B	B	B	No

22. In standard decision tree learning (as explained in the lectures), without pruning. Which would be the first feature chosen for a split?

A x_1 B x_2 C x_3 ✓ D x_4

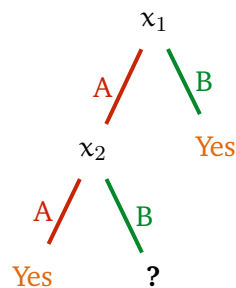
You can work this question out by computing the information gain, but that's slow and error-prone. It's usually quicker to do it intuitively. The best split causes the most uneven distribution over the classes Yes and No on both sides of the split. In this case, x_3 causes only Yes on the A side of the split and only No on the B side (that is, after the split, all instances have the same class and the entropy is zero for both).

23. If we remove that feature from the data, which would be chosen instead?

A x_1 ✓ B x_2 C x_3 D x_4

Note the symmetry in the data. Splitting on x_4 causes a 2/4 distribution on *both* sides of the split. Splitting on x_2 causes a 3/3 distribution and splitting on x_1 causes a 1/5 distribution. If you understand entropy, you won't need the calculator to know that x_1 has the highest information gain.

Consider the following (partial) decision tree:



We can place either x_3 or x_4 on the open node (indicated by the question mark).

24. At this point, what is (approximately) the information gain for x_3 ?
A 0 **B** 0.91 ✓ **C** 1.41 **D** 2.75

For both leaves after splitting on x_3 , there is only one class left, so the entropy is 0 for both. Therefore, the information gain is $H(S) - 0$, where $H(s)$ is the entropy of $S = \{\text{No}, \text{No}, \text{Yes}\}$.

Tip: check your answer against your intuitive understanding of entropy. We know that the answer is equal to $H(S)$, which is the optimum way of encoding a draw from S , or equivalently, a draw from a distribution with $(\frac{1}{3}, \frac{2}{3})$. 1 bit is the entropy for a uniform distribution, so it must be less than that. It also can't be zero, because that would mean we're sure of the outcome.

type: Evidence lower bound

NB: This question type involves filling in the blanks in a derivation. You are free to just memorize the derivation, but note that half the points come from understanding what the derivation means.

The EM and VAE algorithms are both based on the following decomposition.

$$\begin{aligned}
 L(\mathbf{q}, \theta) + \text{KL}(\mathbf{q}, p) &= \mathbb{E}_{\mathbf{q}} \ln \frac{p(\mathbf{x}, \mathbf{z} \mid \theta)}{\mathbf{q}(\mathbf{z} \mid \mathbf{x})} - \mathbb{E}_{\mathbf{q}} \ln \frac{p(\mathbf{z} \mid \mathbf{x}, \theta)}{\mathbf{q}(\mathbf{z} \mid \mathbf{x})} \\
 &= \mathbb{E}_{\mathbf{q}} \ln p(\mathbf{x}, \mathbf{z} \mid \theta) - \mathbb{E}_{\mathbf{q}} \ln \mathbf{q}(\mathbf{z} \mid \mathbf{x}) - \mathbb{E}_{\mathbf{q}} \ln p(\mathbf{z} \mid \mathbf{x}, \theta) + \mathbb{E}_{\mathbf{q}} \ln \mathbf{q}(\mathbf{z} \mid \mathbf{x}) \\
 &= \mathbb{E}_{\mathbf{q}} \ln p(\mathbf{x}, \mathbf{z} \mid \theta) - \mathbb{E}_{\mathbf{q}} \ln p(\mathbf{z} \mid \mathbf{x}, \theta) \\
 &= \mathbb{E}_{\mathbf{q}} \ln \frac{p(\mathbf{x}, \mathbf{z} \mid \theta)}{p(\mathbf{z} \mid \mathbf{x}, \theta)} \\
 &= \mathbb{E}_{\mathbf{q}} \ln \frac{p(\mathbf{z} \mid \mathbf{x}, \theta)p(\mathbf{x} \mid \theta)}{p(\mathbf{z} \mid \mathbf{x}, \theta)} \\
 &= \mathbb{E}_{\mathbf{q}} \ln p(\mathbf{x} \mid \theta) \\
 &= \ln p(\mathbf{x} \mid \theta)
 \end{aligned}$$

Note that while we start with $L(\mathbf{q}, \theta) + \text{KL}(\mathbf{q}, p)$ and end with $\ln p(\mathbf{x} \mid \theta)$, the actual point of this derivation is to simplify $\ln p(\mathbf{x} \mid \theta)$ (which we're actually interested in) into something computable.

25. What should be in place of $\langle \mathbf{a} \rangle$ and $\langle \mathbf{b} \rangle$?

- A** $\langle \mathbf{a} \rangle : \mathbb{E}_{\mathbf{q}} \ln p(\mathbf{z} \mid \mathbf{x}, \theta)$, $\langle \mathbf{b} \rangle : \ln p(\mathbf{x} \mid \theta)$ ✓
B $\langle \mathbf{a} \rangle : \mathbb{E}_{\mathbf{q}} \ln p(\mathbf{z} \mid \mathbf{x}, \theta)$, $\langle \mathbf{b} \rangle : \mathbb{E}_{\mathbf{q}} \ln p(\mathbf{x})$
C $\langle \mathbf{a} \rangle : \mathbb{E}_{\mathbf{q}} \ln p(\mathbf{z}, \mathbf{x} \mid \theta)$, $\langle \mathbf{b} \rangle : \ln p(\mathbf{x} \mid \theta)$
D $\langle \mathbf{a} \rangle : \mathbb{E}_{\mathbf{q}} \ln p(\mathbf{z}, \mathbf{x} \mid \theta)$, $\langle \mathbf{b} \rangle : \mathbb{E}_{\mathbf{q}} \ln p(\mathbf{x})$

26. How is this derivation used in the EM algorithm?

- A To maximize $\ln p(\mathbf{x} \mid \theta)$, we iterate between choosing θ to maximize $L(\mathbf{q}, \theta)$ and then choosing $\mathbf{q}$ to minimize $KL(\mathbf{q}, p)$. ✓
- B To maximize $\ln p(\mathbf{x} \mid \theta)$, we treat $L(\mathbf{q}, \theta)$ as a lower bound and optimize its parameters by backpropagation.
- C To maximize $L(\mathbf{q}, \theta) + KL(\mathbf{q}, p)$ we rewrite it to $\ln p(\mathbf{x} \mid \theta)$ and use random search to find the optimal θ .
- D To maximize $L(\mathbf{q}, \theta) + KL(\mathbf{q}, p)$ we rewrite it to $\ln p(\mathbf{x} \mid \theta)$ and apply the kernel trick to find the optimal θ .

The VAE requires further rewriting. To simplify notation, we will omit the parameters θ .

$$\begin{aligned}
 -\ln p_{\theta}(\mathbf{x}) &\geq -L(\mathbf{q}, \theta) = -\mathbb{E}_{\mathbf{q}} \ln p(\mathbf{x}, \mathbf{z}) + \mathbb{E}_{\mathbf{q}} \ln \mathbf{q}(\mathbf{z} \mid \mathbf{x}) \\
 &= -\mathbb{E}_{\mathbf{q}} \ln p(\mathbf{x} \mid \mathbf{z}) - \mathbb{E}_{\mathbf{q}} \ln p(\mathbf{z}) + \mathbb{E}_{\mathbf{q}} \ln \mathbf{q}(\mathbf{z} \mid \mathbf{x}) \\
 &= -\mathbb{E}_{\mathbf{q}} \ln p(\mathbf{x} \mid \mathbf{z}) + \mathbb{E}_{\mathbf{q}} \ln \frac{\mathbf{q}(\mathbf{z} \mid \mathbf{x})}{p(\mathbf{z})} \\
 &= -\mathbb{E}_{\mathbf{q}} \ln p(\mathbf{x} \mid \mathbf{z}) - \mathbb{E}_{\mathbf{q}} \ln \frac{p(\mathbf{z})}{\mathbf{q}(\mathbf{z} \mid \mathbf{x})} \\
 &= -\mathbb{E}_{\mathbf{q}} \ln p(\mathbf{x} \mid \mathbf{z}) + KL(\mathbf{q}(\mathbf{z} \mid \mathbf{x}), p(\mathbf{z})) \\
 &= -\mathbb{E}_{\mathbf{q}} \ln p(\mathbf{x} \mid \mathbf{z}) + KL(\mathbf{q}(\mathbf{z} \mid \mathbf{x}), \mathcal{N}(\mathbf{0}, \mathbf{I}))
 \end{aligned}$$

27. What should be in place of $\langle \mathbf{a} \rangle$ and $\langle \mathbf{b} \rangle$?

- A $\langle \mathbf{a} \rangle : \ln \mathbb{E}_{\mathbf{q}} p(\mathbf{x}, \mathbf{z}) - \ln \mathbb{E}_{\mathbf{q}} \mathbf{q}(\mathbf{z} \mid \mathbf{x})$, $\langle \mathbf{b} \rangle : \mathcal{N}(\mathbf{0}, \mathbf{I})$
- B $\langle \mathbf{a} \rangle : -\ln \mathbb{E}_{\mathbf{q}} p(\mathbf{x}, \mathbf{z}) + \ln \mathbb{E}_{\mathbf{q}} \mathbf{q}(\mathbf{z} \mid \mathbf{x})$, $\langle \mathbf{b} \rangle : \mathcal{N}(\mathbf{0}, \mathbf{I})$ ✓
- C $\langle \mathbf{a} \rangle : \ln \mathbb{E}_{\mathbf{q}} p(\mathbf{x} \mid \mathbf{z}) - \ln \mathbb{E}_{\mathbf{q}} \mathbf{q}(\mathbf{z} \mid \mathbf{x})$, $\langle \mathbf{b} \rangle : \mathcal{N}(\mathbf{z}, \text{var}(\mathbf{z}))$
- D $\langle \mathbf{a} \rangle : -\ln \mathbb{E}_{\mathbf{q}} p(\mathbf{x} \mid \mathbf{z}) + \ln \mathbb{E}_{\mathbf{q}} \mathbf{q}(\mathbf{z} \mid \mathbf{x})$, $\langle \mathbf{b} \rangle : \mathcal{N}(\mathbf{z}, \text{var}(\mathbf{z}))$

28. Why do we use $-\ln p(\mathbf{x})$ instead of $\ln p(\mathbf{x})$?

- A To give us a reward function (where higher is better) which is the convention in reinforcement learning.
- B To give us a loss function (where lower is better), which is the convention in deep learning systems. ✓
- C To ensure that negative and positive errors don't cancel out against one another.
- D To ensure that negative and positive errors do cancel out against one another.

Maximizing $\ln p_{\theta}(\mathbf{x})$ is a common way of framing the maximum likelihood objective (we want to choose the parameters that maximize the probability (density) of the data we take the logarithm to make the analysis easier or the search more numerically stable). Since deep learning frameworks require a loss to minimize, we minimize $-\ln p_{\theta}(\mathbf{x})$.

type: Naive Bayes

The following dataset represents a spam classification problem: we observe 8 emails and measure two binary features. The first is **T** if the word "pill" occurs in the e-mail (F otherwise) and the second is **T** if the word "meeting" occurs.

"pill"	"meeting"	label
T	F	Spam
T	F	Spam
F	T	Spam
T	F	Spam
F	F	Ham
F	F	Ham
F	T	Ham
T	T	Ham

We build a naive Bayes classifier on this data, as described in the lecture. We estimate the class priors ($p(\text{Spam})$ and $p(\text{Ham})$) from the data.

29. We observe one email that contains both words and one that contains neither. Which class does the classifier assign to each?

- A both words: Ham, neither word: Ham
- B both words: Ham, neither word: Spam
- C both words: Spam, neither word: Ham✓
- D both words: Spam, neither word: Spam

It is extremely easy to make a mistake in these questions, and in this case all possible answers are included. Make sure to check your work carefully.

30. We observe an email e that contains the word "pill", but not the word "meeting". What probabilities does the classifier assign?

- A $p(\text{Ham} | e) = 9/11$, $p(\text{Spam} | e) = 2/11$
- B $p(\text{Ham} | e) = 2/11$, $p(\text{Spam} | e) = 9/11$ ✓
- C $p(\text{Ham} | e) = 9/32$, $p(\text{Spam} | e) = 2/32$
- D $p(\text{Ham} | e) = 2/32$, $p(\text{Spam} | e) = 9/32$

We add pseudo-observations to the data to deal with unseen emails. The pseudo-observations have the same weight as the normal ones. Compute how the judgments change.

NB: We only need to add enough observations to ensure that every value has been seen once per class per feature. So for Spam we add one observation with all F's, one observation with all T's and the same for Ham: four pseudo-observations in total.

“pill”	“meeting”	label
T	F	Spam
T	F	Spam
F	T	Spam
T	F	Spam
F	F	Ham
F	F	Ham
F	T	Ham
T	T	Ham
T	T	Spam
F	F	Spam
T	T	Ham
F	F	Ham

31. We observe one email that contains both words and one that contains neither. Which class does the *smoothed* classifier assign to each?
- A both words: Ham, neither word: Ham
 B both words: Ham, neither word: Spam
 C both words: Spam, neither word: Ham ✓
 D both words: Spam, neither word: Spam
- In a homework exercise, we would make sure that smoothing has an effect, to illustrate its purpose. In an exam, the results are not guaranteed to be so intuitive. Trust your calculations.
32. We observe an email that contains the word “pill”, but not the word “meeting”. What probabilities does the *smoothed* classifier assign?
- A $p(\text{Ham} | e) = 16/22$, $p(\text{Spam} | e) = 6/22$
 B $p(\text{Ham} | e) = 6/22$, $p(\text{Spam} | e) = 16/22$ ✓
 C $p(\text{Ham} | e) = 16/72$, $p(\text{Spam} | e) = 6/72$
 D $p(\text{Ham} | e) = 6/72$, $p(\text{Spam} | e) = 16/72$

type: support vector machines

We have trained a linear support vector machine, and found the following weights and bias:

$$\mathbf{w} = \begin{pmatrix} -3 \\ 2 \end{pmatrix}, \quad b = 2. \quad (5)$$

33. Which of the following points are *support vectors* for the trained model?

- A $y_1 = 1, \mathbf{x}_1 = (1, 1)$ $y_2 = -1, \mathbf{x}_2 = (0, 1)$
 B $y_1 = 1, \mathbf{x}_1 = (1, -1)$ $y_2 = -1, \mathbf{x}_2 = (0, 1)$
 C $y_1 = 1, \mathbf{x}_1 = (1, 1)$ $y_2 = -1, \mathbf{x}_2 = (1, 0)$ ✓
 D $y_1 = 1, \mathbf{x}_1 = (1, -1)$ $y_2 = -1, \mathbf{x}_2 = (1, 0)$

The key here, is to know how a support vector is defined, and what y_i means. This will tell you what the output of the linear classifier function $\mathbf{w}_i^x + b$ should be, and how it should compare to y_i .

34. Consider two points: $\mathbf{x}_1 = (-1, 0)$ and $\mathbf{x}_2 = (0, 1)$. How are these two points classified?

- A Both positive.
- B $\mathbf{x}_1$ is positive, $\mathbf{x}_2$ is negative.
- C $\mathbf{x}_1$ is negative, $\mathbf{x}_2$ is positive. ✓
- D Both negative.

This question is not specific to the support vector machine of the maximum margin hyperplane loss. In all linear classifiers, how $\mathbf{x}$ is classified is based on the sign of $\mathbf{w}^x + b$ in the exact same way.

type: Markov model

We are given a dataset of email, labeled spam and ham. The total number of words in the spam and ham datasets is 50 000 and 15 000, respectively.

On this dataset, we want to train a *first-order Markov model* as a basis for a Bayes classifier to detect spam.

The following table shows the frequency of several bigrams and unigrams in the dataset.

uni- and bigrams	frequency	
	spam	ham
how	3 000	5 000
about	5 000	2 000
a	9 000	9 000
meeting	1 000	3 000
soon	2 000	1 000
how about	500	500
about a	500	200
a meeting	100	300
meeting soon	100	300

We have two *priors* on how likely an email is to be spam. **Prior 1** follows the dataset statistics, which says that an email is twice as likely to be spam as ham. **Prior 2**, based on a user survey, says that the probability that a given email is spam is 1%.

Consider the following email:

email: how about a meeting soon

35. How is the email classified?

- A With both priors as ham. ✓
- B With both priors as spam.
- C Under prior 1 as spam, under prior 2 as ham.
- D Under prior 1 as ham, under prior 2 as spam.

To find the answer you take two steps, first write down the quantities that we're interested in, $p(S | E)$ and $p(H | E)$, using Bayes' rule. Then take the factor $P(E | S)$ (and similar for ham) and apply the chain rule of probability to decompose it into bigram probabilities. Then you can estimate these from the statistics given.

To speed things up, you can do without computing the whole of Bayes' rule. You only need the denominator.

The calculation can be done by hand with a little practice and mental arithmetic, which makes it easier to double-check your work.

36. Under prior 1, what is the probability that the email is spam?

- A $\frac{1}{3}$ B $\frac{1}{6}$ ✓ C $\frac{1}{27}$ D $\frac{1}{270}$

For this you have to use the full version of Bayes' rule. Note however, that if you use the correct formulation, you can mostly re-use terms you already computed in the previous answer.

37. Under prior 2, what is the *probability ratio* of ham over spam?

- A $\frac{1}{990}$ B $\frac{1}{10}$ C 10 D 990 ✓

The *probability ratio* is just the ratio of one probability over another. In this case $p(H | E)/p(S | E)$. If you write this down as a formula first, you'll see that a lot of factors cancel out, and this is a simple fraction of two quantities you've already computed. Note that a ratio of two probabilities can easily be larger than 1.

type: Matrix backpropagation

We have a module f in an automatic differentiation (AD) system (as discussed in the lectures) which computes the following function (its *forward pass*):

$$f_{\mathbf{w}}(\mathbf{x}) = \mathbf{x}^2 \mathbf{w}$$

where $\mathbf{x}$ is a scalar and $\mathbf{w}$ is a vector. Note that this makes f a function from a scalar to a vector.

We will assume that the AD engine will work out the downstream derivatives for us. Given these, we will need to compute the derivatives over the argument $\mathbf{x}$ and over the parameters $\mathbf{w}$. Let the vector $\mathbf{f}$ represent the output of our function.

38. What is the local scalar derivative $\partial f_k / \partial x$?

- A $\partial f_k / \partial x = 2x w_k$ ✓
- B $\partial f_k / \partial x = x w_k$
- C $\partial f_k / \partial x = 2x^2 w_k$
- D $\partial f_k / \partial x = x^2 w_k$

39. Our AD engine provides a vector f^∇ such that $f_i^\nabla = \partial l / \partial f_i$, where l is the loss. What should the module return as the gradients of x ?

- A $x^2 f^\nabla$
- B $2x f^\nabla$
- C $2x f^{\nabla T} w$
- D $x^2 f^{\nabla T} w$

Remember, we're looking to compute $\partial L / \partial x$.

$$\begin{aligned}
 \frac{\partial L}{\partial x} &= \sum_k \frac{\partial L}{\partial f_k} \frac{\partial f_k}{\partial x} && \text{apply mv chain rule} \\
 &= \sum_k \frac{\partial L}{\partial f_k} 2x w_k && \text{from the previous exercise} \\
 &= 2x \sum_k \frac{\partial L}{\partial f_k} w_k \\
 &= 2x \sum_k f_k^\nabla w_k && \text{note that the sum is a dot product, so...} \\
 &= 2x f^{\nabla T} w
 \end{aligned}$$

40. What is the local scalar derivative $\partial f_k / \partial w_i$?

- A $\partial f_k / \partial w_i = x^2$
- B $\partial f_k / \partial w_i = 0$
- C $\partial f_k / \partial w_i = x^2$ if $k = i$, 0 otherwise ✓
- D $\partial f_k / \partial w_i = 0$ if $k = i$, x^2 otherwise

41. Given a vector f^∇ with $f_i^\nabla = \partial l / \partial f_i$, what should the module return as the gradients of w ?

- A $x^2 f^\nabla$
- B $2x f^\nabla$
- C $2x f^{\nabla T} w$
- D $x^2 f^{\nabla T} w$

Remember, we're looking to compute the vector w' where $w'_i =$

$\partial L / \partial \mathbf{w}_i$.

$$\begin{aligned} \frac{\partial L}{\partial \mathbf{w}_i} &= \sum_k \frac{\partial L}{\partial f_k} \frac{f_k}{\partial \mathbf{w}_i} && \text{apply mv chain rule} \\ &= \frac{\partial L}{\partial f_i} \frac{f_i}{\partial \mathbf{w}_i} = \frac{\partial L}{\partial f_i} x^2 && \text{from the previous exercise} \\ &= x^2 f_i^\nabla \end{aligned}$$

Which means that $\mathbf{w}' = x^2 \mathbf{f}^\nabla$.